

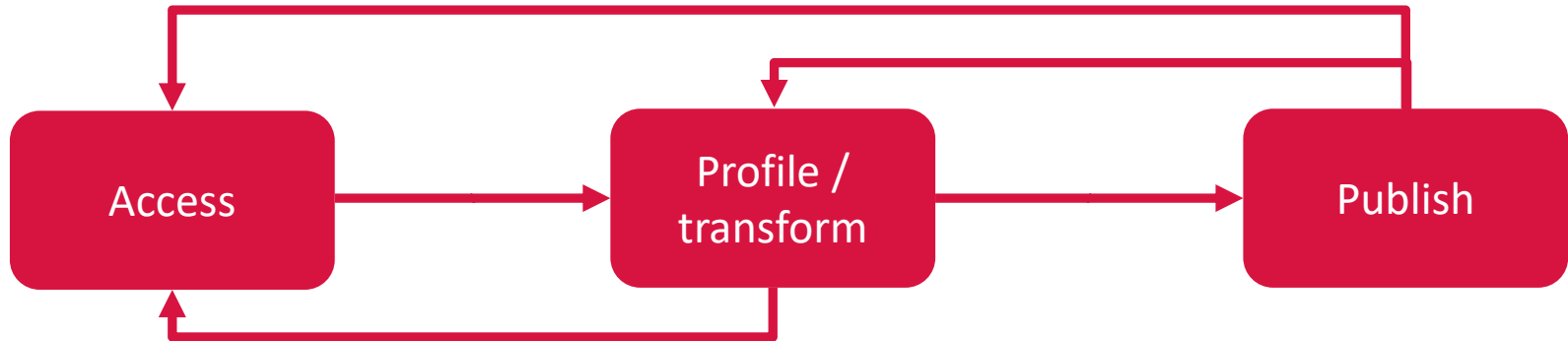
Data Science Fundamentals [SC3021]

Chapter 4: Data Preparation

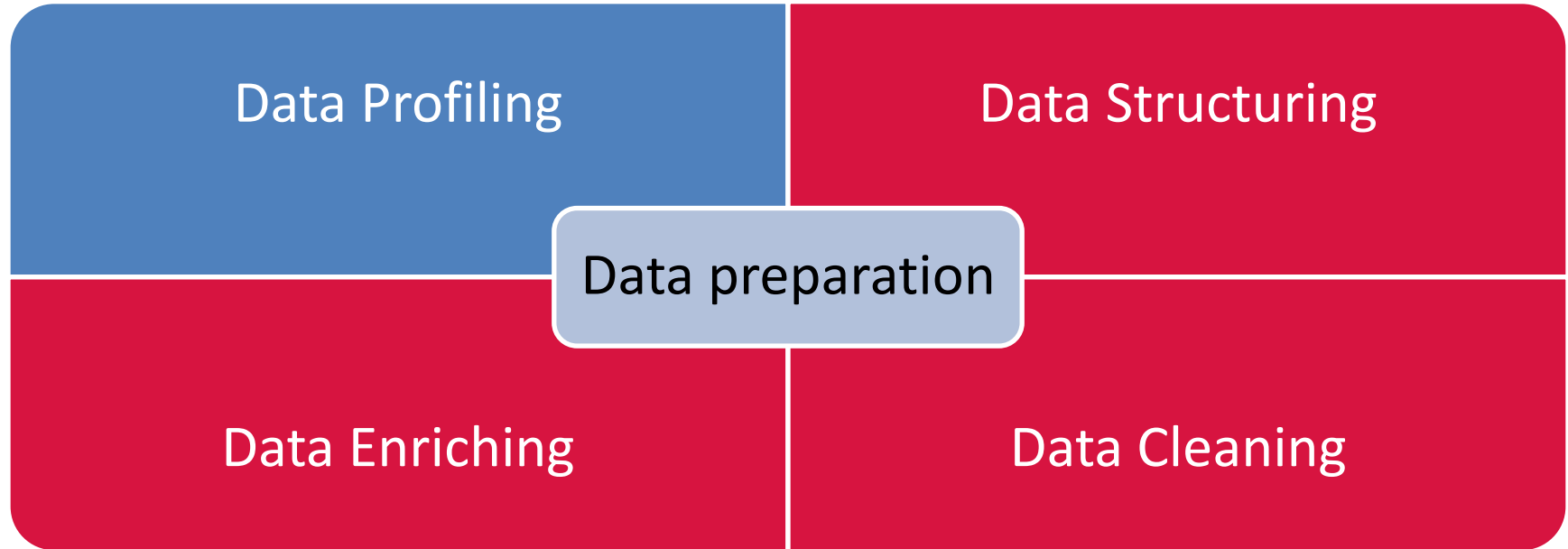
Assoc. Prof. Melanie Herschel | CCDS

Data preparation (reminder)

Data preparation is the process of **converting raw** data into a **form** that allows to extract **value** from data.



Data transformation tasks



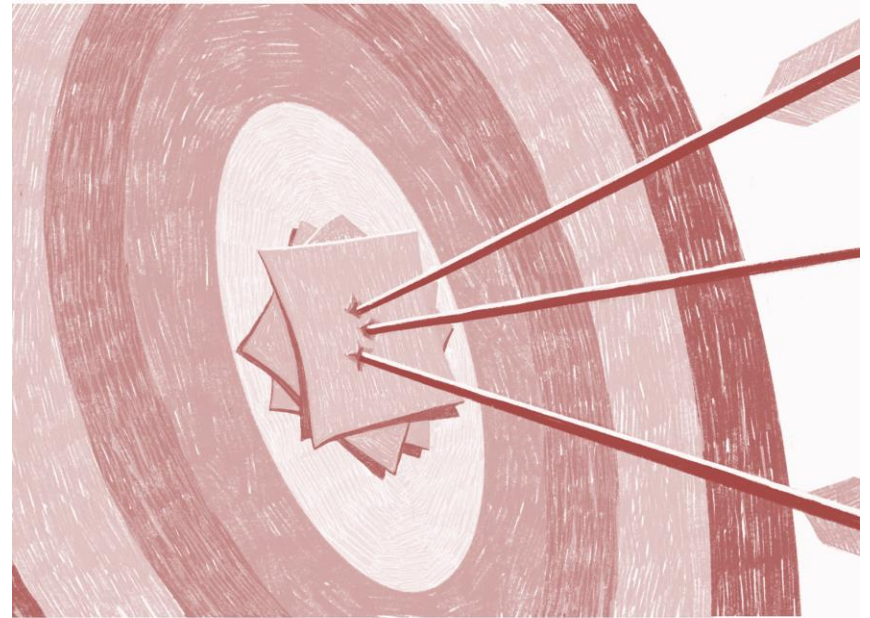


**NANYANG
TECHNOLOGICAL
UNIVERSITY**
SINGAPORE

Chapter 4.3: Data Transformation

At the end of Chapter 4.3, you should be able to...

- ... define and classify various data transformation tasks.
- ... determine which data transformations can be used to prepare the data for a specific data science application.
- ... describe and recognize various data quality issues within and across datasets.
- ... implement various data transformation tasks.



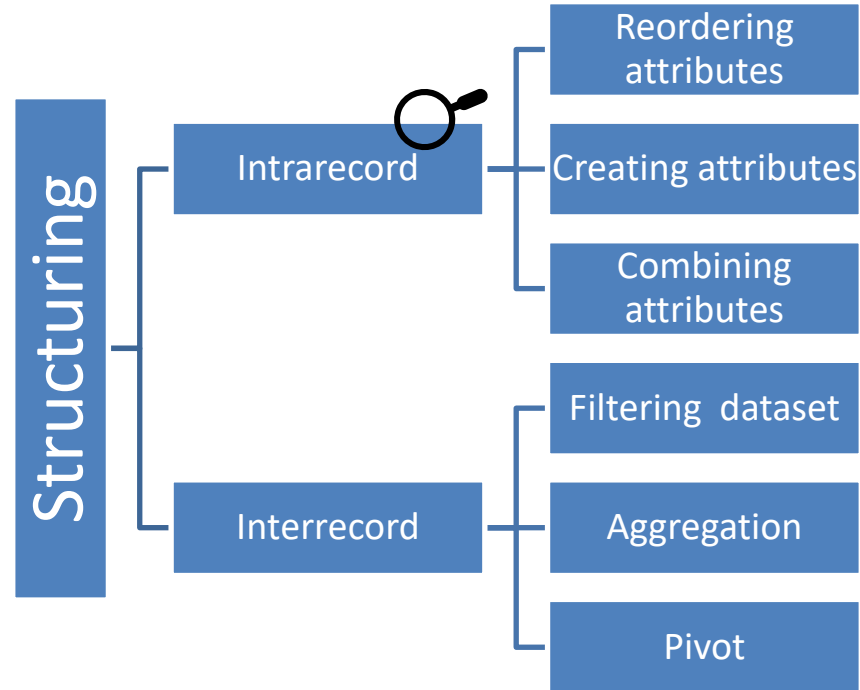
STRUCTURING



Overview of structuring transformations

- Structuring as a transformation action involves **changing the structure and granularity** of a dataset.
 - Changes form and schema of the data.
- Structuring transformations create **new data based on data already present** in the dataset.
- Two main subclasses:
 - **Intrarecord structuring** transformations involve manipulating individual attributes and tuples.
 - **Interrecord structuring** transformations involve tuples and attributes at once.

Overview of structuring transformations



Running example

Example data

- “Contributions by Individuals” file from election year 2015-2016
- Data available at http://www.fec.gov/finance/disclosure/ftpdet.shtml#a2015_2016

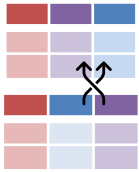
Data dictionary

- Can help understand the permissible values.
- Metadata available at <http://www.fec.gov/finance/disclosure/metadata/DataDictionaryContributionsbyIndividuals.shtml>

Running example – Data excerpt

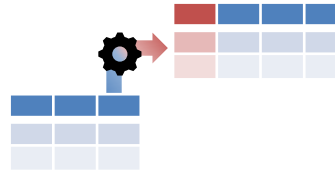
	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
	Column1	Column2	Column3	Column4	Column5	Column6	Column7	Column8	Column9	Column10	Column11	Column12	Column13	Column14	Column15	Column16	Column17
1	C00276311	N	YE	P	2.01601E+17	15	IND	KENNEDY, CHARLIE	SCOTTSBLUFF	NE	69361	BLUE CROSS BLUE SHIELD OF NE	PROVIDER RELATIONSHIP MANAGER	12312016	264		SA0308162
2	C00495358	A	YE	P2018	2.01804E+17	15E	IND	MILLER, LUCILE S	RICHMOND	VA	232263208	N/A	NOT EMPLOYED	12302016	100	C00401224	SA0426188
3	C00495358	A	YE	P2018	2.01804E+17	15E	IND	SCHULZ, ANDREA	NEW YORK	NY	100106410	PENGUIN	EDITOR	12282016	250	C00401224	SA0426188
4	C00495358	A	YE	P2018	2.01804E+17	15E	IND	FINLEY, SHANNON J	WASHINGTON	DC	200071107	CAPITOL COUNSEL, LLC	LOBBYIST	12302016	1700	C00401224	SA0426188
5	C00495358	A	YE	P2018	2.01804E+17	15E	IND	DAKIN, WILLIAM G	WASHINGTON	DC	200082663	N/A	RETIRED	12272016	100	C00401224	SA0426188
6	C00495358	A	YE	P2018	2.01804E+17	15E	IND	LESTER, HARRY T	VIRGINIA BEACH	VA	234552848	EASTERN VIRGINIA MEDICAL SCHOOL	PRESIDENT	12292016	2700	C00401224	SA0426188
7	C00495358	A	YE	P2018	2.01804E+17	15E	IND	EVANS, MEGAN	ALEXANDRIA	VA	223143821	N/A	NOT EMPLOYED	12302016	500	C00401224	SA0426188
8	C00495358	A	YE	P2018	2.01804E+17	15E	IND	KNAPP, JOHN W SR	LEXINGTON	VA	244501789	N/A	NOT EMPLOYED	12272016	50	C00401224	SA0426188
9	C00495358	A	YE	P2018	2.01804E+17	15E	IND	MANUS, JILLIAN	ATHERTON	CA	940275440	STRUCTURE CAPITAL	VENTURE CAPITALIST	12272016	500	C00401224	SA0426188
10	C00495358	A	YE	P2018	2.01804E+17	15E	IND	CORBETT, LEO	NEW YORK	NY	100295245	SELF EMPLOYED	BUSINESS CONSULTANT	12282016	100	C00401224	SA0426188
11	C00495358	A	YE	P2018	2.01804E+17	15E	IND	ALTSCHULER, SHARMAN	CAMBRIDGE	MA	21383329	MOONBOX PRODUCTIONS	THEATER PRODUCER	12302016	500	C00401224	SA0426188
12	C00495358	A	YE	P2018	2.01804E+17	15E	IND	DAVID, JOSEPH J	CHARLOTTESVILLE	VA	22902	SELF-EMPLOYED	PHYSICIAN	12102016	250	C00401224	SA0426188
13	C00495358	A	YE	P2018	2.01804E+17	15E	IND	MISKA, SIMA	PINECREST	FL	331566105	N/A	HOMEMAKER	12312016	50	C00401224	SA0426188
14	C00495358	A	YE	P2018	2.01804E+17	15E	IND	WALDRON, KAREN HOLLY	SHAWSVILLE	VA	241621840	FRALIN & WALDRON INC	REAL ESTATE DEVELOPER	12192016	100	C00401224	SA0426188
15	C00495358	A	YE	P2018	2.01804E+17	15E	IND	WARNER, RAY	WILLIAMSBURG	VA	231853976	SMITH DAWSON & ANDREWS	GOVERNMENT RELATIONS	12212016	250	C00401224	SA0426188
16	C00495358	A	YE	P2018	2.01804E+17	15E	IND	CROSWELL, CLYDE	BERRYVILLE	VA	226112408	N/A	NOT EMPLOYED	12102016	25	C00401224	SA0426188
17	C00495358	A	YE	G2018	2.01804E+17	15	IND	STONE, CATHLEEN D	BOSTON	MA	21081103	SELF EMPLOYED	ATTORNEY	12232016	2500		SA0426188
18	C00495358	A	YE	G2018	2.01804E+17	15	IND	MASON, CYNTHIA	CHESTER	VA	238314700	N/A	RETIRED	12302016	500		SA0426188
19	C00495358	A	YE	P2018	2.01804E+17	15E	IND	DAKIN, WILLIAM G	WASHINGTON	DC	200082663	N/A	RETIRED	12302016	100	C00401224	SA0426188
20	C00495358	A	YE	P2018	2.01804E+17	15E	IND	THIBODEAUX, JOYCE	HOUMA	LA	703605932	N/A	NOT EMPLOYED	12272016	25	C00401224	SA0426188
21	C00495358	A	YE	P2018	2.01804E+17	15E	IND	HAUSLER, RICHARD W	FAIRFAX	VA	220312711	INSIGHT PROPERTY GROUP LLC	EXECUTIVE/ATTORNEY	12082016	500	C00401224	SA0426188
22	C00495358	A	YE	P2018	2.01804E+17	15E	IND	KAUFMAN, SUSAN F	VIRGINIA BEACH	VA	234541624	N/A	NOT EMPLOYED	12102016	250	C00401224	SA0426188
23	C00495358	A	YE	P2018	2.01804E+17	15E	IND	KAY, SAUL	ENCINO	CA	913164415	N/A	RETIRED	12142016	100	C00401224	SA0426188
24	C00495358	A	YE	P2018	2.01804E+17	15E	IND	REEDER, JAMES	FRANKLIN	VA	238512863	TRAVEL INCORPORATED	TRAVEL AGENT	12302016	100	C00401224	SA0426188
25	C00495358	A	YE	P2018	2.01804E+17	15E	IND	HUFFMAN, ROBERT K	MCLEAN	VA	221012239	AKIN GUMP STRAUSS HAUER & FELD	LAWYER	12302016	25	C00401224	SA0426188
26	C00495358	A	YE	P2018	2.01804E+17	15E	IND	OMOTAYO, ADEWALE	NORTH CHESTERFIELD	VA	232363511	BIG O'CAB	TRANSPORTER	12302016	25	C00401224	SA0426188
27	C00495358	A	YE	P2018	2.01804E+17	15E	IND	THIBODEAUX, JOYCE	HOUMA	LA	703605932	N/A	NOT EMPLOYED	12292016	50	C00401224	SA0426188
28	C00495358	A	YE	P2018	2.01804E+17	15E	IND	MURPHY, ARLENE	FAIRFIELD	CT	68246503	N/A	COMMUNITY VOLUNTEER	12312016	200	C00401224	SA0426188
29	C00495358	A	YE	P2018	2.01804E+17	15E	IND	CROSWELL, CLYDE	BERRYVILLE	VA	226112408	N/A	NOT EMPLOYED	12312016	100	C00401224	SA0426188
30	C00495358	A	YE	P2018	2.01804E+17	15E	IND	THIBODEAUX, JOYCE	HOUMA	LA	703605932	N/A	NOT EMPLOYED	12202016	50	C00401224	SA0426188
31	C00495358	A	YE	P2018	2.01804E+17	15E	IND	ALTSCHULER, SHARMAN	CAMBRIDGE	MA	21383329	MOONBOX PRODUCTIONS	THEATER PRODUCER	12082016	25	C00401224	SA0426188
32	C00495358	A	YE	P2018	2.01804E+17	15E	IND	FREY, RUSSELL	MCLEAN	VA	221013206	EDHELPER	PRESIDENT	12252016	1000	C00401224	SA0426188
33	C00495358	A	YE	P2018	2.01804E+17	15E	IND	SHEEHAN, ROBERT	NEW YORK	NY	100257659	SKADDEN ARPS SLATE MEAGHER&FLOM	ATTORNEY	12312016	100	C00401224	SA0426188

Intrarecord structuring



Reordering attributes

Attributes and values remain unchanged



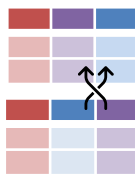
Creating attributes

Attributes are added with values **extracted** from existing attributes



Combining attributes

Attributes are **merged** into a single attribute



Reordering attributes

	A	B	C	D	
1	Column1	Column2	Column3	Column4	Column5
2	C00276311	N	YE		2.01
3	C00495358	A	YE	2018	2.01
4	C00495358	A	YE	2018	2.01
5	C00495358	A	YE	2018	2.01
6	C00495358	A	YE	2018	2.01
7	C00495358	A	YE	2018	2.01
8	C00495358	A	YE	2018	2.01
9	C00495358	A	YE	2018	2.01
10	C00495358	A	YE	2018	2.01
11	C00495358	A	YE	2018	2.01
12	C00495358	A	YE	2018	2.01
13	C00495358	A	YE	2018	2.01
14	C00495358	A	YE	2018	2.01
15	C00495358	A	YE	2018	2.01
16	C00495358	A	YE	2018	2.01
17	C00495358	A	YE	2018	2.01

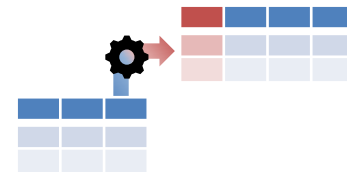
Reordering attributes

We switch columns 2 and 3 in the example file.

```
[67] # switch columns based on their initial positions in the original schema
```

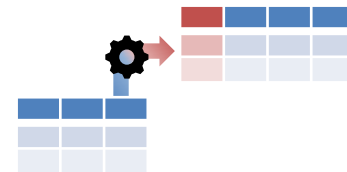
```
df = df[ df.columns.tolist()[0:1] +      #keep first attribute (Column 1)
         df.columns.tolist()[2:3] +      # add third attribute (Column 3)
         df.columns.tolist()[1:2] +      #add second attribute (Column 2)
         df.columns.tolist()[3:]]        #append any remaining attributes
print(df.columns)
```

```
Index(['Column1', 'Column3', 'Column2', 'Column4', 'Column5', 'Column6',
      'Column7', 'Column8', 'Column9', 'Column10', 'Column11', 'Column12',
      'Column13', 'Column14', 'Column15', 'Column16', 'Column17', 'Column18',
      'Column19', 'Column20', 'Column21'],
      dtype='object')
```



Extracting Values

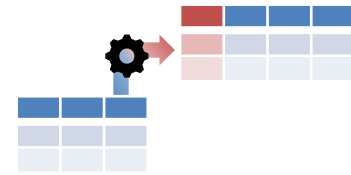
- Extraction involves **creating a new column** (i.e., attribute and values for this new attribute) **from an existing column**.
- Three common extraction approaches:
 - Positional extraction
 - Pattern extraction
 - Complex structure extraction



Extracting values – Positional extraction

- Involves identifying a substring in an existing column C and placing that substring into a new column C'.
- Requires the specification of positions (start index, end index) to delimit the substring to be extracted from values of C and stored in the corresponding value of C'.
- Common use:
 - Date-time fields
 - Fixed-width fields

Extracting values – Positional extraction



Column14
12312016
12302016
12282016
12302016
12272016
12292016
12302016
12272016
12272016

- This column stores a data in the format MMDDYYYY.
- We extract three attributes from it: month, day, year.
- Fixed positions of extracted values throughout the column.

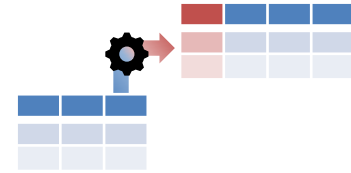
Positional extraction

From Column 14, extract values for new attributes based on fixed positions in the fixed length string describing a date in MMDDYYYY format.

```
df['month'] = df['Column14'].astype(str).str[:2]  
df['day'] = df['Column14'].astype(str).str[2:4]  
df['year'] = df['Column14'].astype(str).str[4:]  
print(df[['Column14', 'month', 'day', 'year']])
```

	Column14	month	day	year
0	12312016	12	31	2016
1	12302016	12	30	2016
2	12282016	12	28	2016
3	12302016	12	30	2016
4	12272016	12	27	2016

Extracting values – Positional extraction



Column8
KENNEDY, CHARLIE
MILLER, LUCILE S
SCHULZ, ANDREA
FINLEY, SHANNON J
DAKIN, WILLIAM G
LESTER, HARRY T
EVANS, MEGAN
KNAPP, JOHN W SR
MANUS, JILLIAN
CORBETT, LEO
ALTSHULER, SHARMAN
DAVID, JOSEPH J
MISKA, SIMA

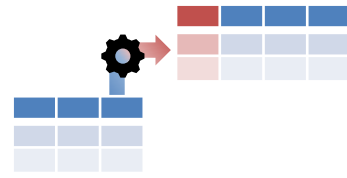
- This column stores strings representing LASTNAME, FIRSTNAMES.
- We can separate them via the position of the comma (,)
- Position of separator varies and has to be determined for extraction

```
# Find the position of the comma in 'Column8'
df['comma_pos'] = df['Column8'].str.find(',')

# Extract last name and first name (while coping with Column8 values without ,)
df['firstname'] = df.apply(lambda row:
    row['Column8'][int(row['comma_pos']) + 1:].lstrip()
    if row['comma_pos'] > -1 else None, axis=1)
df['lastname'] = df.apply(lambda row:
    row['Column8'][:int(row['comma_pos'])].rstrip()
    if row['comma_pos'] > -1 else None, axis=1)

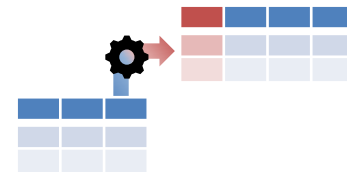
# Display the results
print(df[['Column8', 'lastname', 'firstname']])
```

	Column8	lastname	firstname
0	KENNEDY, CHARLIE	KENNEDY	CHARLIE
1	MILLER, LUCILE S	MILLER	LUCILE S
2	SCHULZ, ANDREA	SCHULZ	ANDREA
3	FINLEY, SHANNON J	FINLEY	SHANNON J
4	DAKIN, WILLIAM G	DAKIN	WILLIAM G
...	...	...	...
662545	MIGNANO, JOHN E. MD, PHD	MIGNANO	JOHN E. MD, PHD
662546	PACKIANATHAN, SATYASEELAN MD, PHD	PACKIANATHAN	SATYASEELAN MD, PHD
662547	PAWLICKI, TODD PHD	PAWLICKI	TODD PHD
662548	PRESTIDGE, BRADLEY D. MD, MS	PRESTIDGE	BRADLEY D. MD, MS



Extracting values – Pattern extraction

- Uses rules to describe the sequence of characters or patterns to be extracted.
- Such patterns can frequently be specified by regular expressions (regex).
- Some regex notation
 - `\d` matches a digit
 - `\s` matches a whitespace
 - `*` for 0 or more occurrences
 - `?` For 0 or one occurrence
 - `|` either or
 - `[0-9]` accepted range of values
- Full reference: [Python RegEx](#)



Extracting values – Pattern extraction

Column20
P/R DEDUCTION (\$97.00 MONTHLY)
P/R DEDUCTION (\$25.00 MONTHLY)
P/R DEDUCTION (\$75.00 MONTHLY)
P/R DEDUCTION (\$36.55 MONTHLY)
P/R DEDUCTION (\$20.00 MONTHLY)
P/R DEDUCTION (\$45.00 MONTHLY)
P/R DEDUCTION (\$20.00 MONTHLY)
P/R DEDUCTION (\$60.00 MONTHLY)
P/R DEDUCTION (\$20.00 MONTHLY)
P/R DEDUCTION (\$25.00 MONTHLY)
P/R DEDUCTION (\$25.00 MONTHLY)
P/R DEDUCTION (\$25.00 MONTHLY)
P/R DEDUCTION (\$20.00 MONTHLY)
P/R DEDUCTION (\$25.00 MONTHLY)
P/R DEDUCTION (\$15.00 SEMI-MONTHLY)
P/R DEDUCTION (\$30.00 MONTHLY)
P/R DEDUCTION (\$42.11 SEMI-MONTHLY)



- We want to create a new column
MONTHLY_CONTRIBUTION
- Needs extraction of the digits + dot + digits between the \$-sign and the keyword MONTHLY.

Regular expression:
`\$\d+.\d*\sMONTHLY`

Extracting values – Pattern extraction

Regular expression:
`\$\d+.\?\d*\sMONTHLY`

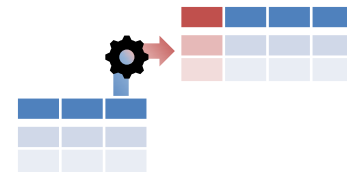
- Regular expressions integrated in `re` Python library
- `search()` returns all substrings of text that match the expression (can be more than one).

```
import re

def extract_contribution(txt):
    # Search for the pattern
    # $ + digits + . (optional) + digits (optional) + space + MONTHLY
    match = re.search(r"\$\d+.\?\d*\sMONTHLY", str(txt))
    if match:
        # Remove MONTHLY suffix and $ prefix
        return float(match.group(0).split(' ')[0][1:])
    else:
        return np.NaN

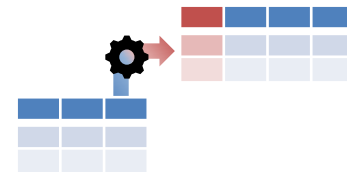
df['MONTHLY_CONTRIBUTION'] = df['Column20'].apply(extract_contribution)
print(
    df[['Column20', 'MONTHLY_CONTRIBUTION']].
    sort_values(by='MONTHLY_CONTRIBUTION', ascending=False).head(100)
)
```

	Column20	MONTHLY_CONTRIBUTION
3454	P/R DEDUCTION (\$437.50 MONTHLY)	437.50
16316	P/R DEDUCTION (\$416.66 MONTHLY)	416.66
26381	P/R DEDUCTION (\$416.66 MONTHLY)	416.66
14576	(\$416.66 MONTHLY)	416.66



Extracting values – Complex structures

- Data is not always structured in tables.
- Semi-structured data allows complex hierarchical structures (e.g., XML, JSON)
- Two types of complex structures in JSON, namely JSON array and JSON map



Complex structures in JSON

JSON array $[e_1, e_2, \dots, e_n]$

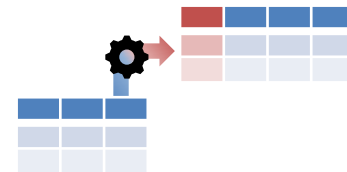
- Ordered sequence of elements e_i
- Elements in arrays can be of simple or complex type, or null

```
[  [1, "Alice", 3],  
    ["Bob"],  
    [ [true, false], 8, 9]  
]
```

JSON map $\{k_1:v_1, \dots, k_n:v_n\}$

- Set of key-value pairs $k_i: v_i$
- Key k_i is a string representing a property name and value v_i represents the value of that property.

```
{  "courseID": "SC3021",  
    "weeks": [  
        {  "weekID": 1,  
            "lecture": "Intro",  
            "tutorial": "project"  
        }, ... ]  
}
```



Extracting values – Complex structures

- Semi-structured data allows variable structure across records.
 - Different array lengths
 - Varying properties across records
 - etc.

```
{ "courseID": "SC3021",  
  "weeks": [  
    { "weekID":1,  
      "lecture":"Intro",  
      "tutorial": "project"    }, ...] }
```

- Extraction from these complex structures necessary to obtain tabular data.

CourseID	weekID	Lecture	Tutorial	Lab
SC3021	1	Intro	Project	NULL
...	...	...	...	...
SC3021	4	Profiling	Ecosystem	Task1

Extracting values – Complex structures

```
import json

# Create a list to store the JSON records
json_records = []

# Create example data

data = { "courseID": "SC3021",
        "weeks": [{ "weekID":1,"lecture":"Intro", "tutorial": "project"},
                  { "weekID":4,"lecture":"Profiling", "tutorial": "Ecosystem", "Lab":"Task1"} ]
        }

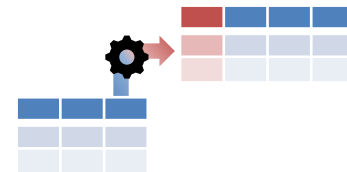
# Append the dictionary to the list
json_records.append(data)

# Convert the list of dictionaries to a JSON string
json_data = json.dumps(json_records, indent=4)

# Convert the JSON string back to a list of dictionaries
json_records = json.loads(json_data)

# Create a table that extracts all values
df = pd.json_normalize(json_records[0], 'weeks', ['courseID'])
print(df)
```

	weekID	lecture	tutorial	Lab	courseID
0	1	Intro	project	NaN	SC3021
1	4	Profiling	Ecosystem	Task1	SC3021



When to use which extraction technique

Positional extraction

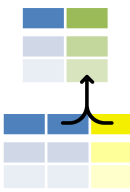
- When each record field conforms to a known structure, and each unique substring in the field always begins and ends at the same position
- When each record field contains multiple substrings that are separated by the same delimiter string. Substrings do not necessarily need to be the same length or conform to the same pattern.

Pattern-based extraction

- When you want to extract a substring that can be defined by a generic pattern.

Complex structure extraction

- When your record fields contain JSON maps or arrays, and you want to extract individual elements from those structures. Many datasets that are machine-generated contain complex JSON structures.



Combining multiple attributes

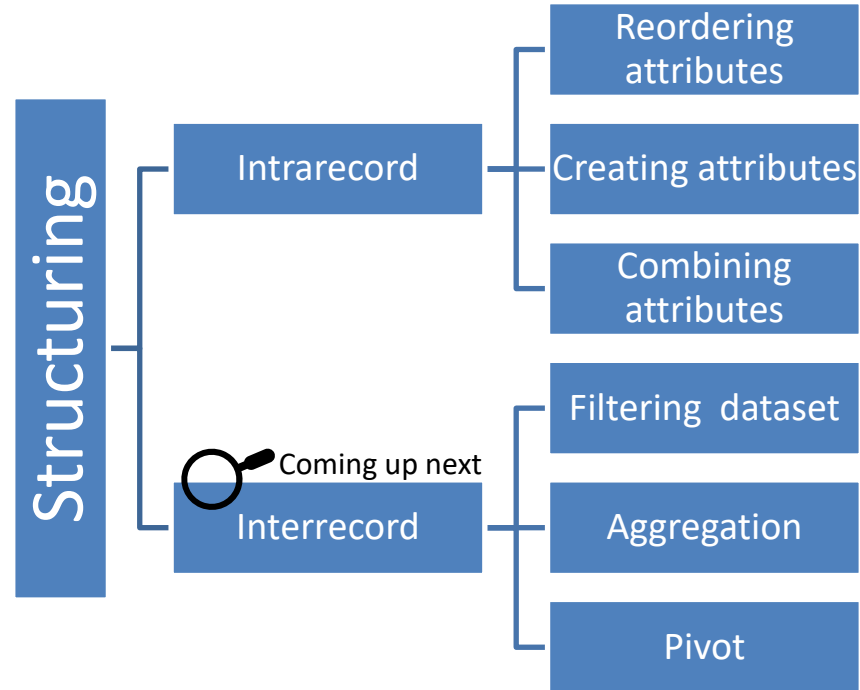
Column9	Column10
SCOTTSBLUFF	NE
RICHMOND	VA
NEW YORK	NY
WASHINGTON	DC
WASHINGTON	DC
VIRGINIA BEACH	VA
ALEXANDRIA	VA
LEXINGTON	VA
ATHERTON	CA
NEW YORK	NY
CAMBRIDGE	MA



```
# Combine 'Column9' and 'Column10'  
df['Combined_Column'] = df['Column9'] + ', ' + df['Column10']  
print(df[['Column9', 'Column10', 'Combined_Column']])
```

	Column9	Column10	Combined_Column
0	SCOTTSBLUFF	NE	SCOTTSBLUFF, NE
1	RICHMOND	VA	RICHMOND, VA
2	NEW YORK	NY	NEW YORK, NY
3	WASHINGTON	DC	WASHINGTON, DC
4	WASHINGTON	DC	WASHINGTON, DC
...	...	...	...
662545	BROOKLINE	MA	BROOKLINE, MA
662546	JACKSON	MS	JACKSON, MS
662547	LA JOLLA	CA	LA JOLLA, CA
662548	NORFOLK	VA	NORFOLK, VA
662549	APPLETON	WI	APPLETON, WI

Overview of structuring transformations



References and credits

- Joseph M. Hellerstein, Tye Rattenbury, Jeffrey Heer, Sean Kandel, Connor Carreras. Principles of Data Wrangling. O'Reilly Media. 2017 – Chapter 4